Module 5 – Hands-on Sessions

CS5013 – Programming with AI
Testing and Documentation with AI

V. Krishna Nandivada
Department of Computer Science and Engineering, IIT Madras

Overview

Module 5 has two hands-on sessions:

- **Session 5A – AI-generated tests with a coverage target and a mutation check.** Take an under-tested class, generate unit tests with an LLM, verify they raise coverage *and* that mutation testing kills the mutants.
- **Session 5B – Auto-documentation.** Take an undocumented class, generate JavaDoc, README, and an OpenAPI spec. Verify every claim in the docs against the code.

Both use a common starter under `m5-pub/`.

Starter contents (`m5-pub/`)

- `TaxCalculator.java` – ∼60-line class with five public methods; existing suite has line coverage ≈ 45% and branch coverage ≈ 30%.
- `TaxCalculatorTest.java` – three seed tests.
- `OrderApi.java` – Spring `@RestController` with four endpoints; no JavaDoc, no README section.
- `OrderDto.java`, `Order.java`, `OrderService.java` – supporting types.
- `Makefile` – pre-configured with JUnit 5, JaCoCo, PIT.

1 Session 5A: AI-generated tests

Objectives

By the end of this session you will be able to:

1. Measure coverage with JaCoCo and interpret its report.
2. Prompt an AI to generate JUnit 5 tests in AAA style.
3. Recognise and delete tautological or implementation- assertion tests.
4. Run PIT on your suite and interpret the mutation report.
5. Add one test that specifically kills a surviving mutant.

Setup

1. Download https://www.cse.iitm.ac.in/~krishna/cs5013/m5-pub.tgz.
2. `cd m5-pub`.
3. Run `make coverage`.
4. Open the report at `build/reports/jacoco/test/html/index.html`. Note the numbers.

Part A – Baseline coverage

Record in `PROMPTS.md`:

- Line coverage percentage.
- Branch coverage percentage.
- Which methods have $< 50\%$ branch coverage?

Part B – Generate tests with constraints

Pick the method with the lowest branch coverage. Paste its source into chat with the prompt:

```
Generate JUnit 5 tests for the following method.
Rules:
- Use AAA style with explicit comments.
- One assertion per test.
- Test names of the form
  <method>_<condition>_<expected>.
- Cover: (a) null input, (b) empty input if applicable,
  (c) boundary values, (d) one representative happy path.
- Do NOT use mocks.
- Do NOT include implementation-copying assertions; each
  assertion must state a contract, not restate the code.
```

Save the AI reply in `PROMPTS.md`. Paste the generated tests into `TaxCalculatorTest.java`. Run `make test` and confirm they pass.

Part C – Read every assertion

Now the important step. For each generated assertion, ask: *does it encode a contract, or does it restate the code?*

Common pitfall

An easy way to spot an implementation-copying assertion: the assertion's expected value is a computation from the input that matches what the method would compute. If your "expected" is `a * 0.18 + a` and the method returns `price * 0.18 + price`, the assertion asserts the implementation. Delete it or reword the expected as a domain-derived value.

Delete or rewrite any assertion that fails the "does this encode a contract" question.

Part D – Mutation testing

Run `make mutation`. Open the report at `build/reports/pitest/index.html`.

For your chosen method, note:

- Mutation score (killed / total).
- List of surviving mutants.

Pick one surviving mutant. Paste the mutant description into chat with:

```
The mutant at line <N> (‘<original>‘ -> ‘<mutated>‘) survived
the test suite. Write ONE new JUnit test that would kill this
mutant without changing any other test. Explain in one
sentence why this test kills the mutant.
```

Add the test. Re-run PIT. Confirm the mutant is now killed.

Part E – Reflect

In `PROMPTS.md`, record:

- Line and branch coverage after your changes (compared to baseline).
- Mutation score before and after the targeted test.
- One sentence on whether the mutation-testing feedback felt more useful, or less useful, than the coverage feedback.

Deliverables

- `PROMPTS.md`

2 Session 5B: Auto-documentation

Objectives

By the end of this session you will be able to:

1. Generate JavaDoc that describes contracts, not implementations.
2. Verify JavaDoc against the method body with an AI cross-check.
3. Draft a README with an anti-hallucination escape hatch.
4. Generate an OpenAPI 3.0 spec from a Spring controller and verify it renders in Swagger UI.

Setup

Continue on the same branch. Ensure `OrderApi.java` has no JavaDoc yet.

Part A – JavaDoc for one endpoint

Pick one endpoint on `OrderApi` (e.g., `POST /orders`). Paste the method into chat with:

```
Generate JavaDoc for the following method. Rules:
- One-line summary in imperative mood.
- Describe the contract, not the implementation.
- @param for each parameter with type and constraints.
- @return with what is returned.
- @throws for each declared or unchecked exception the
  caller should be aware of.
- If any behaviour is not evident from the code, write
  "TODO" and skip.
```

Paste the JavaDoc into the source.
Now the verification step:

```
Given the JavaDoc above and the method body below,
identify any statement in the JavaDoc that is inconsistent
with the code. Do not fix; just list.
```

Fix any inconsistencies the AI flagged. Repeat until the verification step returns "no inconsistencies".

Part B – Draft the README

Paste the file tree and top-level files (with truncation if needed) into chat with:

```
Draft a README.md for this repository with sections:
description, build, quick example, contributing, license.
Use MIT license placeholder.
Rules:
- Do NOT invent features not present in the code.
- If a section has no evidence in the code, write "TODO" and
  skip.
- The one-line description must be a factual summary of what
  the code does, not marketing copy.
```

Save the raw draft in `README.raw.md`.
Now edit the raw draft:

- Delete any invented features (verify by grepping the code for the claimed capability).
- Fill in the one-line description in your own words.
- Replace any TODO section for which you can write real content by hand.

Save the edited version as `README.md`. In `PROMPTS.md`, list the invented features you deleted (if any).

Part C – OpenAPI spec

Paste the full `OrderApi.java` into chat with:

```
Read the following Spring @RestController and generate an
OpenAPI 3.0 YAML spec covering:
- Every endpoint (path, method, summary from JavaDoc).
- Request body schemas for POST/PUT.
- Response schemas for 2xx and 4xx.
- Referenced DTO schemas in the components section.
```

```
If any endpoint's behaviour is unclear, add a TODO comment
in the spec at that location.
```

Save the output as `openapi.yaml`.
Render it: paste it into editor.swagger.io (or run Swagger UI locally). Verify:

- Every endpoint appears with the correct path and method.
- Every referenced DTO is defined in the components section.
- Every method has at least one 2xx response and at least one 4xx response.

Add any missing 4xx responses by hand.

Part D – Reflect

In `PROMPTS.md`, answer:

- How many JavaDoc inconsistencies did the AI's self-check catch on the first pass?
- How many invented features did the README pass produce?
- Which docs artefact (JavaDoc, README, OpenAPI) needed the most hand editing, and why?

Deliverables

Amend the PR from Session 5A to also include:

- JavaDoc on the chosen endpoint.
- `README.raw.md` and `README.md`.
- `openapi.yaml`.
- Updated `PROMPTS.md` covering both sessions.